Chapitre 7

Sécurité des bases de données

Dr. Wafa Berrayana

2025

Introduction à la sécurité des bases de données

- Actuellement, les données sont au cœur des organisations : **informations clients**, **données financières**, **dossiers médicaux**, **inventaires**, **statistique, …**

  ➔ Leur compromission peut entraîner des conséquences graves : **pertes économiques, atteintes à la vie privée, sanctions légales, réputation affectée,...**

- **La sécurité des bases de données** regroupe **les technologies**, **politiques** et **processus mis en place** pour protéger les données stockées contre l'accès non autorisé, la manipulation ou la destruction
- **La sécurité des bases de données** vise à garantir **la confidentialité**, **l'intégrité** et **la disponibilité** des informations en utilisant des mesures comme le chiffrement, la gestion des accès, l'audit et la surveillance.

Objectifs principaux de la sécurité des bases de données (1)

- **Confidentialité :** Empêcher l'accès aux données par des personnes non autorisées

- **Intégrité :** Assurer que les données sont exactes et fiables, et qu'elles n'ont pas été altérées de manière non autorisée

- **Disponibilité :** Garantir que les données et les systèmes de base de données sont accessibles aux utilisateurs autorisés lorsque c'est nécessaire

Objectifs principaux de la sécurité des bases de données (2)

- **Maintien de la confidentialité**
  - Il s'agit de détecter ou d'empêcher des accès non autorisés
  - C'est crucial :
    - dans des environnements critiques ou stratégiques : militaires ou commerciaux, par exemple.
    - pour respecter le droit des individus à décider comment et dans quel but les informations les concernant peuvent être extraites, mémorisées ou transmises à d'autres individus.
- **Maintien de l'intégrité**
  - Il s'agit de détecter ou d'empêcher des modifications illicites des données qu'elles soient dues à :
    - des pannes de système,
    - des manipulations erronées,
    - des sabotages

Objectifs principaux de la sécurité des bases de données (3)

- **Maintien de la disponibilité**
  - Il s'agit de détecter ou d'empêcher des **dénis de service**.
  - Il y a déni de service lorsqu'un utilisateur ne parvient pas à accéder dans un **délai raisonnable**, à une information ou à une ressource pour laquelle il a une autorisation d'accès.
  - Par exemple, une attaque consistant à saturer un serveur de fausses requêtes empêchant les requêtes valides d'être exécutées.

Attaques

- Les violations de la sécurité d'une BD consistent en des lectures ou des mises à jour illicites.
- Les événements qui portent ces violations sont appelées des **attaques**
- On distingue :
  - les attaques **non frauduleuses** :
    - catastrophes naturelles,
    - pannes de logiciel ou de matériel,
    - erreurs humaines …
  - les attaques **frauduleuses** :
    - utilisation abusive de leurs droits par les utilisateurs,
    - agents hostiles exécutant des actions de destruction du logiciel ou du matériel, ou lisant ou mettant à jour des données protégées,
    - ces agents peuvent être cachés dans des actions légales : **chevaux de Troie**.

Les principaux risques liés aux bases de données (1)

- Les **bases de données** sont exposées à plusieurs **menaces** qui peuvent **compromettre leur sécurité**
  - **Les injections SQL** représentent une des attaques les plus fréquentes : les cybercriminels exploitent des failles dans les **requêtes SQL mal protégées** pour accéder aux données ou les modifier.

    ➔ Ces attaques sont souvent dues à **une absence de validation des entrées utilisateur** dans les **systèmes connectés aux bases**

  - **La mauvaise configuration** constitue une autre source de risque. Des autorisations trop larges, des bases accessibles directement sur Internet ou l'absence de pare-feu **augmentent les chances de piratage**. Un système mal configuré est une porte d'entrée idéale pour des hackers expérimentés

Les principaux risques liés aux bases de données (2)

- Les **cyberattaques ciblées**, comme les ransomwares ou les **attaques DDoS**, peuvent également **paralyser une base de données.**
  - Les **ransomwares** cryptent les données et demandent une rançon pour les libérer, tandis que les attaques **DDoS** saturent les serveurs, rendant les bases inaccessibles

- Les **erreurs humaines** restent un facteur majeur de vulnérabilité.
  - Une **mauvaise manipulation**, comme la suppression accidentelle de données critiques ou l'utilisation de mots de passe faibles, peut facilement **compromettre la sécurité des systèmes**

Exemples d'impacts des failles de sécurité sur les entreprises

- Les **violations de données** peuvent coûter des millions d'euros à une entreprise, non seulement en termes financiers, mais aussi en termes de **crédibilité.** Selon une étude IBM, le **coût moyen d'une violation de données** en 2023 a atteint **4,45 millions de dollars**

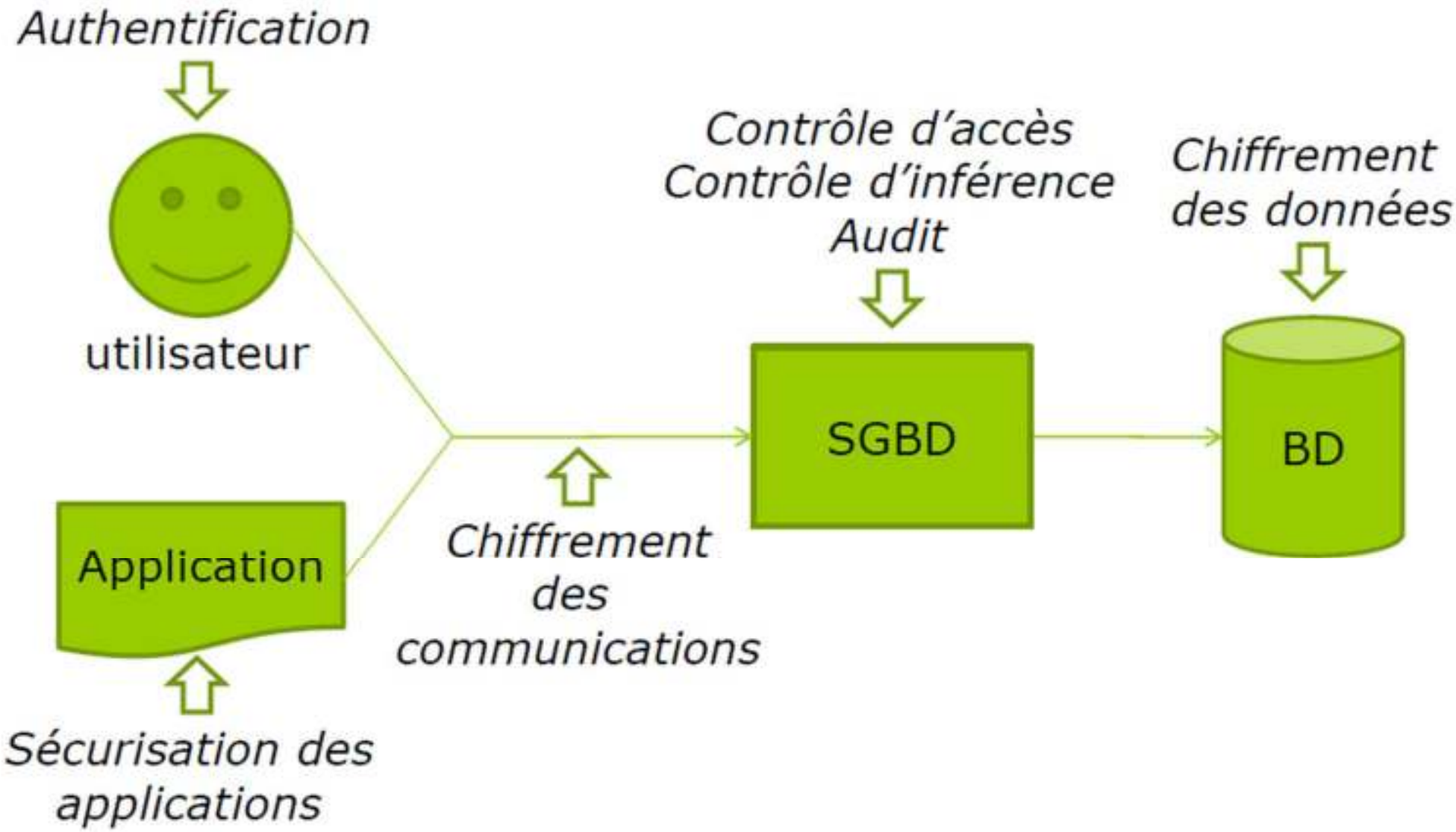
- une société e-commerce a dû **fermer temporairement son site web après un vol massif d'informations bancaires.** Ce temps d'arrêt a engendré **une perte de chiffre d'affaires considérable**

- une banque européenne **a déboursé des millions** pour **sécuriser ses systèmes** après une intrusion ayant compromis les informations bancaires de ses clients

Outils utilisés pour la sécurisation et le contrôle de conformité des bases de données

2025

Protections contre les attaques



Protections contre les attaques: Authentification

- **L'authentification** a pour objectif d'assurer que l'utilisateur qui se connecte à la BD :
  - est autorisé à se connecter,
  - est bien celui qui s'annonce.

- **L'authentification** peut reposer sur :
  - la sécurité des mots de passe
  - des techniques d'identification biométriques
  - ..

Protections contre les attaques: chiffrement (1)

- **Le chiffrement** d'une base de données transforme les données lisibles en un format crypté, accessible uniquement aux personnes disposant de la clé de déchiffrement. Il protège la **confidentialité** cet l'**intégrité** des informations stockées, même en cas d'accès non autorisé ou de fuite
- **Méthodes courantes** de chiffrement
  - **Chiffrement symétrique** (**AES** le plus répandu) & **chiffrement asymétrique**
  - **Chiffrement au niveau de la base** : chiffre l'intégralité de la base ou du fichier (ex. Transparent Data Encryption – TDE).
  - **Chiffrement au niveau des colonnes :** chiffre uniquement les colonnes sensibles (ex. Always Encrypted pour SQL Server, TDE pour Oracle).
  - **Chiffrement réseau :** protège les données lors de leur transfert entre le client et la base.

Protections contre les attaques: chiffrement (2)

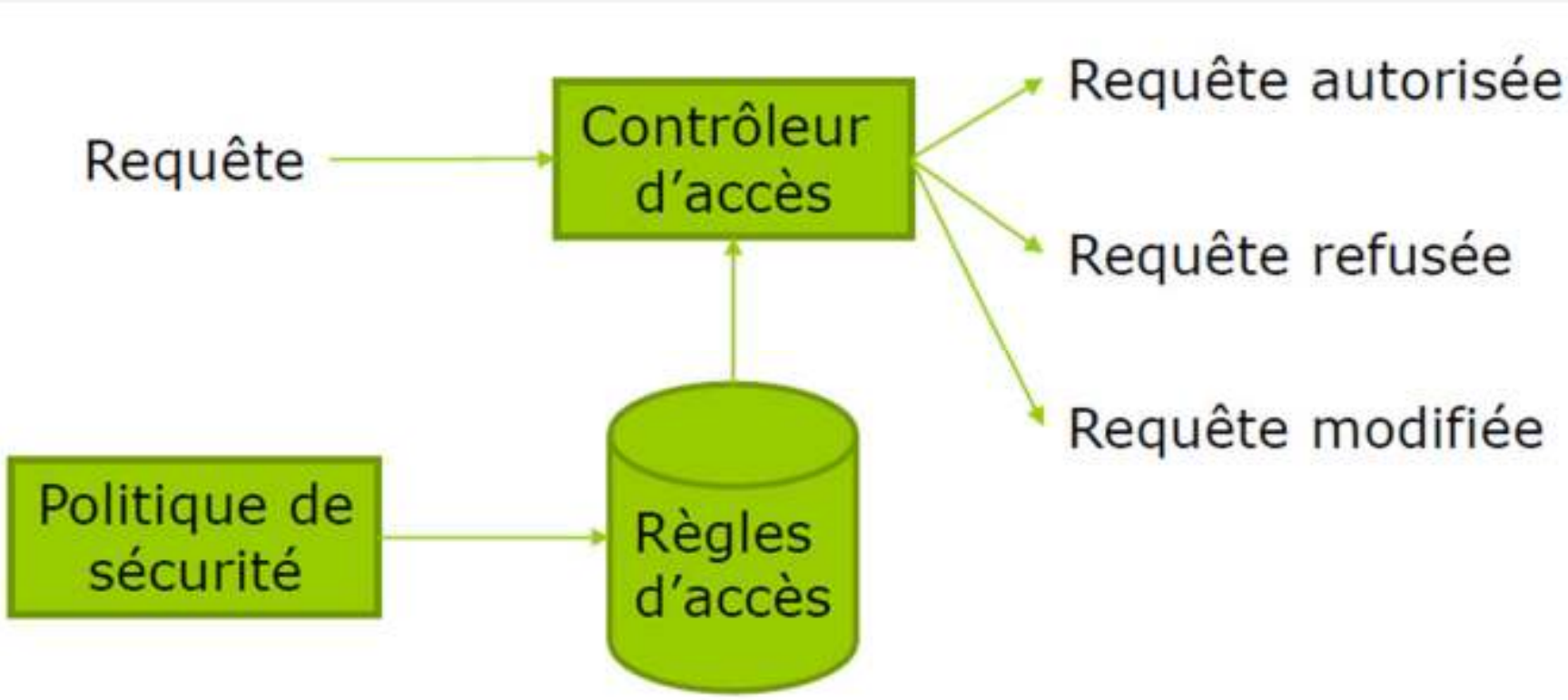
- Le **hachage** est utilisé dans les bases de données pour sécuriser certaines données, principalement les **mots de passe** et garantir l'intégrité des informations. Il consiste à transformer une donnée en une chaîne de caractères unique (empreinte ou hash), qui ne peut pas être inversée pour retrouver la donnée d'origine
- Rôle du **hachage** dans une base de données
  - **Sécurité des mots de passe :** Les mots de passe sont stockés sous forme hachée, ce qui empêche de les retrouver en cas de fuite de la base.
  - **Contrôle d'intégrité :** Le hachage permet de vérifier qu'une donnée n'a pas été modifiée, car toute modification change l'empreinte.
  - **Indexation rapide :** Les hachages sont parfois utilisés pour accélérer la recherche dans de grandes bases de données

Protections contre les attaques: Contrôle d'accès

- Un système de **contrôle d'accès** comprend :
  - des **sujets** : utilisateurs, processus… qui peuvent être classés par groupes,
  - des **objets** : données, programmes…
  - des **opérations** : lecture, ajout, modification, suppression…, **déclenchées par les sujets sur les objets**
  - un **règlement de sécurité** constitué d'un ensemble de **règles d'accès** traduisant la **politique de sécurité** du système d'information.
  - un **processeur de sécurité** qui vérifie que les requêtes adressées au système ne violent pas les règles d'accès et selon le cas autorise, modifie ou interdit la requête.

Politique de sécurité & Processeur de sécurité

- Les ITSEC définissent la **politique de sécurité** comme l'ensemble des *lois règles ou pratiques qui régissent la façon dont l'information sensible et les autres ressources sont gérées, protégées et distribuées à l'intérieur d'un système d'information*
- **Processeur de sécurité**



Principe du moindre privilège

- Ce principe stipule qu'un sujet ne doit disposer que des droits d'accès minimum pour assurer l'exécution des tâches qui lui sont assignées, pas un de plus.
- Ex : ne pas donner les droits de l'administrateur à tout utilisateur d'un système (système d'exploitation, SGBD).

Autorisation, interdiction et obligation

- Les règlements les plus simples ne contiennent que des **autorisations** :
  - ce qui n'est pas autorisé est interdit.

- Certains règlements incluent des **interdictions** afin de spécifier des exceptions à des permissions générales.
  - Ex : les patients ont droit de consulter leur dossier médical sauf Jean Dupont.

- D'autres enfin, plus sophistiqués, incluent des **obligations** :
  - difficiles à implanter dans les systèmes informatiques

Modèles de contrôle d'accès

- On distingue deux grandes catégories de modèles de contrôle d'accès :

  - **les modèles discrétionnaires** :
    - Le créateur d'un objet en est son propriétaire et le transfert des privilèges sur ces objets est à sa discrétion.

  - **les modèles obligatoires** :
    - Il s'agit de protéger le secret et l'intégrité.
    - Les sujets (ou utilisateurs) et objets sont classés par niveaux (ex: Top secret, Secret, Confidentiel, Public).

Le contrôle d'inférence (1)

- Le **contrôle d'inférence** est un mécanisme de sécurité utilisé dans les **bases de données**, surtout les bases statistiques ou contenant des données sensibles (santé, finance…)
- Il sert à **empêcher un utilisateur d'inférer** une information confidentielle à partir de données **non sensibles** ou **agrégées**

- **But :** empêcher qu'on déduise un secret même quand on n'a pas un accès direct aux données confidentielles

Le contrôle d'inférence (2)

- **Exemple simple**
  - On empêche l'accès direct au salaire d'Ali. Mais l'utilisateur peut exécuter :
    - Moyenne des salaires du département IT
    - Moyenne des salaires du département A sans Ali
    - En comparant les deux moyennes, il **retrouve le salaire d'Ali.**
  - **Requête 1**

```
SELECT AVG(salaire) FROM Employes WHERE departement = 'IT';
```

  - **Requête 2**

```
SELECT AVG(salaire) FROM Employes WHERE departement = 'IT' AND nom <> 'Ali';
```

C'est une attaque d'inférence.

L’audit d’une base de données

- C’est un processus systématique de surveillance et d’enregistrement des activités qui s’y déroulent. Il vise à faire un :
  - **audit** des entrées dans la base,
  - **audit** des utilisations de la BD en dehors des heures ouvrables,
  - **audit** de la manipulation du schéma,
  - **audit** des erreurs,
  - **audit** des modifications des sources des procédures stockées et des triggers,
  - **audit** des modifications des attributs de sécurité (login, privilèges...)

Les attaques à une BD

- Les attaques à une BD peuvent exploiter les failles des applications opérant sur cette BD :
  - stockage des mots de passe dans les fichiers de configuration de l'application
  - scripts de connexion à la BD accessibles dans le code source de l'application,
    - attaques par injection SQL,
    - attaques exploitant les débordements de tampons
    - ...

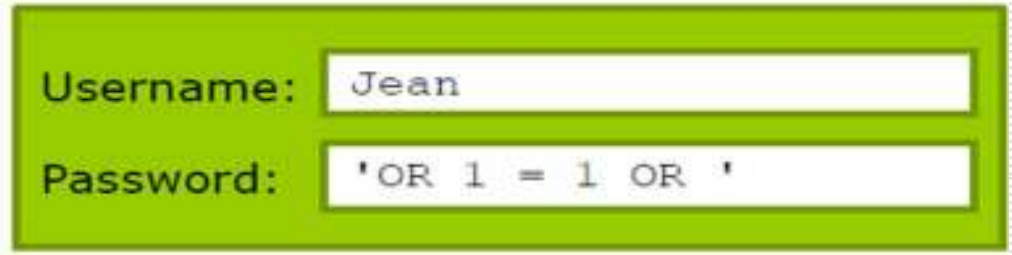
Les attaques à une BD : Injection SQL (1)

- On suppose qu'une application récupère le nom *username* et le mot de passe *password* saisi, dans un formulaire, par un utilisateur et se connecte à la BD en exécutant la requête affectée à la variable **$requete** composée par concaténation :

```
$requete =
"SELECT *
 FROM users
 WHERE username = '" & username & "'
 AND password = '" & password & "' ;"
```

Les attaques à une BD : Injection SQL (2)

- Un attaquant désirant se connecter comme administrateur pourra saisir dans le formulaire :



- La requête générée sera :

```
SELECT *
FROM users
WHERE username = 'Jean'
AND password = '' OR true OR '';
```

- L'attaquant se retrouvera connecté comme 1er utilisateur de la BD qui a toutes les chances d'être l'administrateur

Modèles de contrôle d'accès

2025

Contrôle d'accès obligatoire (MAC: Mandatory Access Control)

- Ce mode est basé sur des niveaux de classification (Sécret, Confidentiel, Public).
- L'accès dépend de l'**étiquette de sécurité** (label) affectée à l'utilisateur et à l'objet.
- Utilisé dans :
  - Milieu militaire
  - Institutions gouvernementales
  - Systèmes nécessitant une forte confidentialité

- **Avantage :** sécurité très élevée
- **Inconvénient :** rigidité, difficile à administrer

Contrôle d'accès discrétionnaire (DAC: Discretionary Access Control) (1)

- C'est le mode le plus **flexible**.
- Le **propriétaire** d'un objet (table, vue, fichier…) décide qui peut accéder à quoi.
- Gestion avec des permissions : `SELECT`, `INSERT`, `UPDATE`, `DELETE`, etc.
- Exemple :
  - L'utilisateur Ahmed donne à l'utilisateur Sara le droit de lire une table :
  - `GRANT SELECT ON` Clients `TO` Sara;
- **Avantage :** flexible et simple
- **Inconvénient :** plus vulnérable au partage incontrôlé (propagation de droits)

Contrôle d'accès discrétionnaire (DAC: Discretionary Access Control) (2)

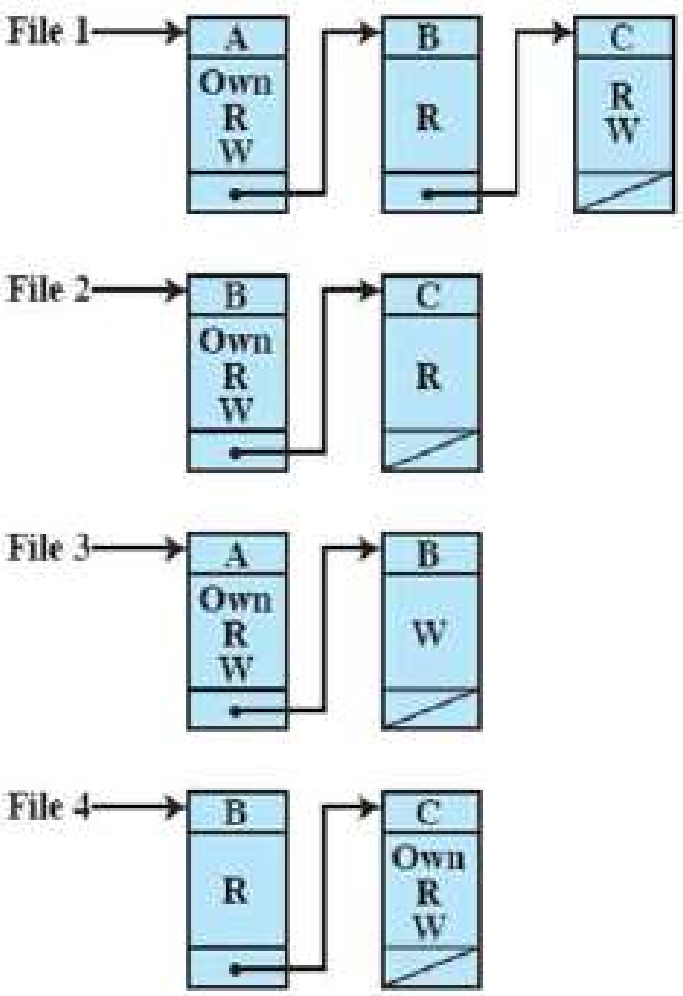
- DAC est fondamentalement basé sur une matrice de contrôle d'accès

- Dans ce modèle, la matrice représente les **sujets** (utilisateurs ou processus) en lignes et les **objets** (fichiers, bases de données, etc.) en colonnes, avec les **droits d'accès** (lecture, écriture, exécution) dans les cellules

- Le propriétaire de l'objet décide discrétionnairement des autorisations et peut les déléguer, ce qui est implémenté via:
  - **une ACL (Access Control Lists)** ou
  - **une matrice d'accès** indiquant les modes d'accès à un objet autorisés pour un sujet

Access Control List Vs matrice d'accès

- une ACL (Access Control Lists)



- une matrice d'accès

	File 1	File 2	File 3	File 4
User A	Own R W		Own R W	
User B	R	Own R W	W	R
User C	R W	R		Own R W

Matrice d'accès (1)

- Pour un modèle basé sur l'utilisation d'une matrice d'accès, le système peut être défini par le quadruplet **(S, O, R, M)** :
  - ***S*** est l'ensemble des ***sujets***,
  - ***O*** est l'ensemble des ***objets***,
  - ***R*** est l'ensemble des ***modes d'accès***,
  - ***M*** est la ***matrice d'accès***

Matrice d'accès (2)

- **Sujet**
  - Un sujet est une entité active qui peut exécuter des actions et de qui le système doit être protégé.
  - Un sujet peut être un utilisateur, un ensemble d'utilisateurs, un programme...

- **Objet**
  - Un objet est une entité qui doit être protégée.
  - Un objet est soit :
    - une entité passive (fichiers, programmes, base de données...),
    - un sujet

Matrice d'accès (3)

- **Modes d'accès**
  - read (**r**)
    - lecture d'une information contenue dans un objet
  - write (**w**)
    - écriture d'une information dans un objet en voyant son contenu
  - append (**a**)
    - ajout d'une information à un objet sans voir son contenu
  - execute (**x**)
    - exécution d'un programme
  - **own**
    - possession d'un objet

Matrice d'accès (4)

- **Matrice d'accès**
  - ***M*[*s*, *o*]** = ensemble des opérations autorisées pour le **sujet *s*** sur **l'objet o** (ensemble des **privilèges** de *s* sur *o*)
  - Si **own** ∈ *M*[*s*, *o*] alors *s* est le propriétaire de *o* et l'administrateur des autorisations sur *o* :
    - il peut les transmettre à d'autre utilisateurs

Exemple de matrice d'accès

sujets \ objets	fichier F_1	fichier F_2
utilisateur U_1	**own** U_1 a tous les droits sur F_1 dont celui de transmettre tout ou partie de ces droits à un autre utilisateur	**x*** U_1 a le droit d'exécuter F_2 et de transmettre ce droit à un autre utilisateur
utilisateur U_2	–	**own**
programme P	**r, w** P a le droit de lire ou de modifier F_1	–

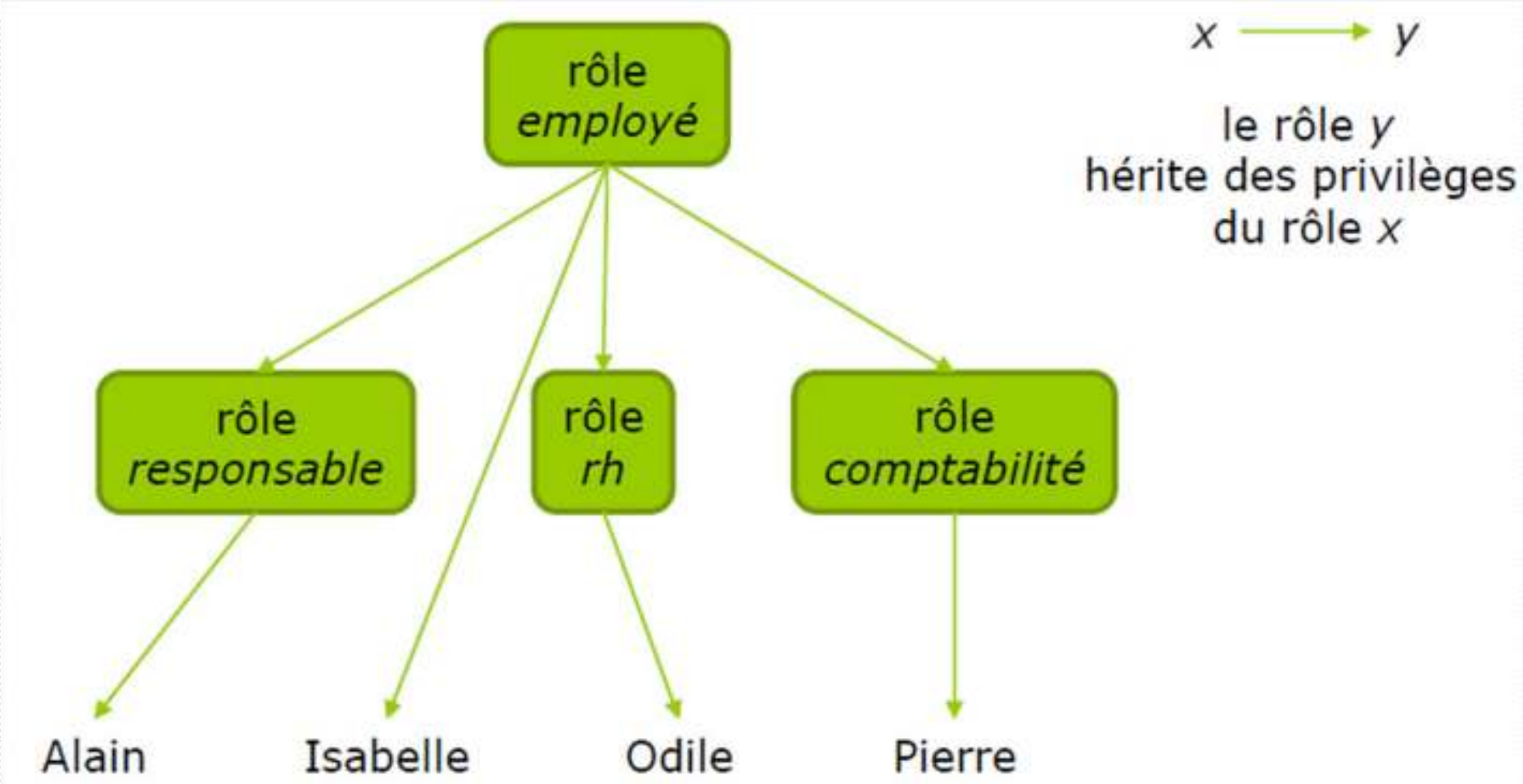
Contrôle d'accès basé sur les rôles (RBAC: Role-Based Access Control)

- Les permissions ne sont pas attribuées directement aux utilisateurs, mais à des **rôles**.
- Exemple de rôles :
  - **Admin, Comptable, Analyste, Utilisateur**.
- Les utilisateurs reçoivent ensuite un ou plusieurs rôles.
- Exemple :

```
CREATE ROLE Analyste;
GRANT SELECT ON Transactions TO Analyste;
GRANT Analyste TO Wafa;
```

- **Avantage :** facile à gérer pour les grandes organisations
- **Inconvénient :** nécessite de bien planifier les rôles

Contrôle d'accès basé sur les rôles : Exemple



Contrôle d'accès basé sur les attributs (ABAC: Attribute-Based Access Control)

- Les droits sont accordés en fonction d'**attributs** :
  - Attributs de l'utilisateur (poste, département…)
  - Attributs de l'objet
  - Contexte (heure, localisation)

- Exemple :
  - Un analyste financier peut accéder aux rapports *uniquement* pendant les heures de bureau

- **Avantage :** très flexible et dynamique
- **Inconvénient :** plus complexe à mettre en œuvre

Contrôle d'accès basé sur des règles (RB: Rule-Based Access Control)

- L'accès dépend de **règles prédéfinies**. Exemples :
  - Blocage automatique après 5 tentatives échouées
  - Accès interdit les week-ends
  - Filtrage basé sur IP

- **Avantage :** simple pour appliquer des politiques globales
- **Inconvénient :** moins personnalisé pour chaque utilisateur

Contrôle d’accès en SQL

2025

MySQL et modèle de contrôle d'accès

- MySQL adopte **principalement le mode de contrôle d'accès discrétionnaire DAC.** Depuis MySQL 8.0, il intègre aussi un **RBAC simplifié**, mais **le modèle central reste le DAC**

- **MySQL n'utilise pas :**
  - **MAC** (mandatory access control)
  - **ABAC** (attribute-based access control)
  - **Rule-Based** avancé
  - **Label Security**

SGBD	Modèle dominant
MySQL	✔ DAC + RBAC simplifié
Oracle	✔ RBAC avancé + MAC + ABAC (via policies)

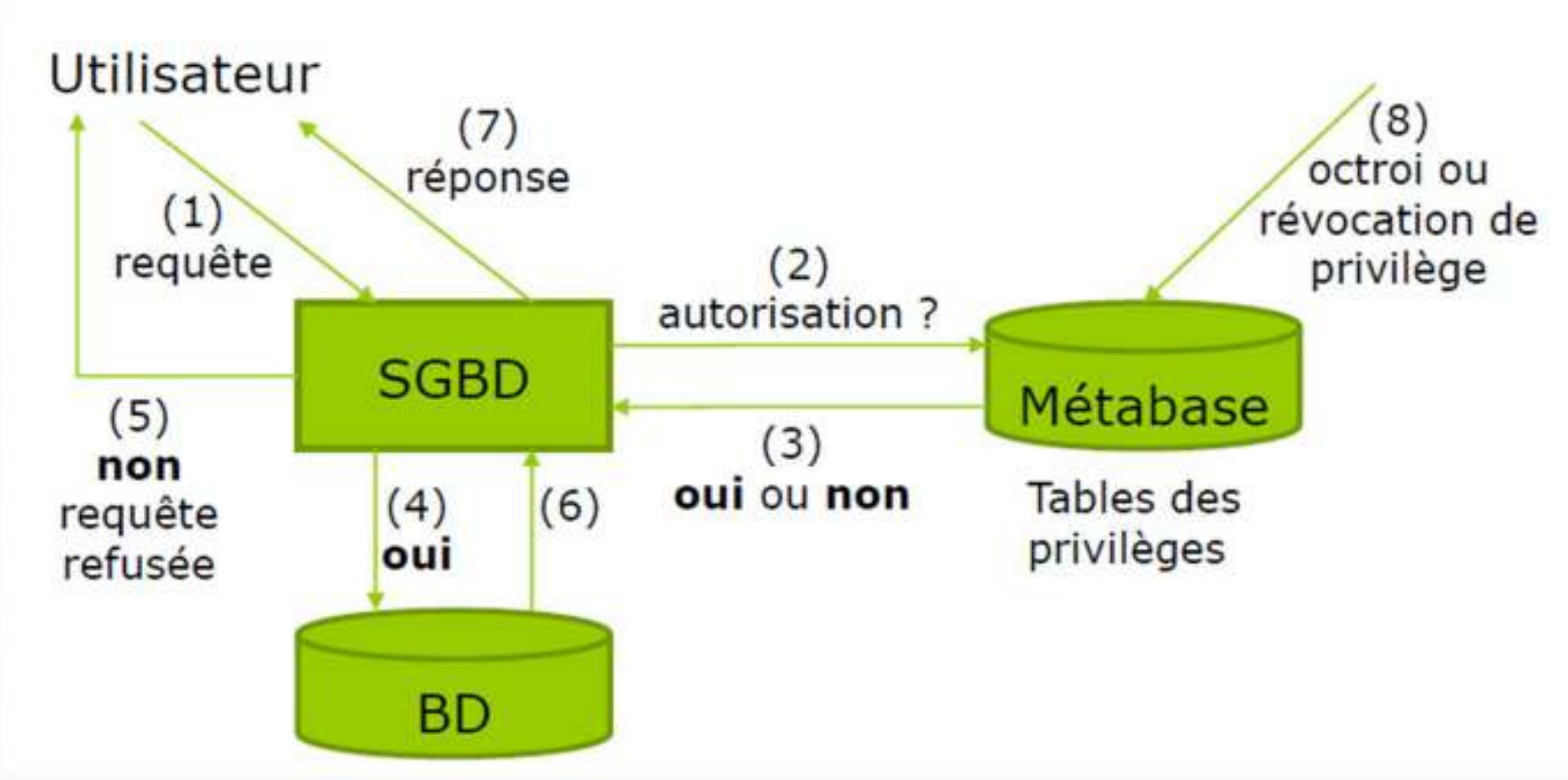
Sujets, objets, privilèges et rôles

- **Sujets**
  - utilisateurs, groupe d'utilisateurs, tous les utilisateurs
- **Objets**
  - BD, tables, vues, index, procédures…
- **Privilèges (ou autorisations)**
  - sur les tables,
  - sur le schéma,
  - sur la base de données…
- **Rôles**
  - groupes de privilèges

Modèle de contrôle d'accès

- Le modèle de contrôle d'accès SQL est un modèle discrétionnaire :
  - Le créateur d'un objet possède tous les privilèges sur cet objet.
  - Pour pouvoir accéder à un objet qu'il n'a pas créé, un utilisateur doit avoir reçus les privilèges nécessaires

Processus de contrôle



Création des utilisateurs

- La création des utilisateurs est du ressort de l'administrateur de la base de données qui peut créer :
  - des utilisateurs,
  - des groupes d'utilisateurs

- SQL ne spécifie pas de commande pour la création des utilisateurs :
  - cette commande est donc spécifique à chaque SGBD.

Privilèges

- On distingue :
  - les **privilèges objets** qui concernent des opérations précises sur des tables, des vues, des procédures stockées…, dont le nom est spécifié ;
  - les **privilèges systèmes** qui concernent des opérations sur tous les objets d'une certaine catégorie

- Ces privilèges varient sensiblement d'un SGBD à l'autre.

Privilèges objets (1)

- **SELECT SELECT(*c1, …, cn*)**
  - pour pouvoir lire le contenu de toutes ou de certaines colonnes d'une table.
- **INSERT INSERT(c1, …, cn)**
  - pour pouvoir insérer une valeur dans toutes ou certaines colonnes d'une table.
- **UPDATE UPDATE(c1, …, cn)**
  - pour pouvoir modifier le contenu de toutes ou de certaines colonnes d'une table.
- **DELETE**
  - pour pouvoir supprimer des lignes d'une table.

Privilèges objets (2)

- **`REFERENCES REFERENCES(c1, …, cn)`**
  - pour pouvoir faire référence à une table ou à certaines colonnes d'une table dans une contrainte d'intégrité.
- **`TRIGGER`**
  - pour pouvoir placer un trigger sur une table.
- **`USAGE`**
  - pour pouvoir manipuler les domaines.
- **`EXECUTE`**
  - pour pouvoir exécuter une procédure stockée

Privilèges système

- ORACLE, par exemple, propose un jeu très complet de privilèges systèmes (plus d'une centaine) qui permettent de réaliser des opérations sur tous les objets d'une certaine catégorie

- Par exemple :
  - **`INSERT ANY TABLE`**
  - **`DELETE ANY INDEX`**

Octroi ou révocation de privilèges

- SQL offre deux commandes pour octroyer ou révoquer des privilèges :
  - **GRANT**
  - **REVOKE**

GRANT

- Syntaxe

```
GRANT (liste de privilèges | ALL)
ON liste d'objets
TO liste d'utilisateurs | PUBLIC
[WITH GRANT OPTION]
```

- avec :
  - **ALL :** tous les privilèges que le donneur peut accorder
  - **PUBLIC** : tous les utilisateurs connus du système
  - **WITH GRANT OPTION :** indique que le receveur pourra transmettre les privilèges qui lui sont octroyés

REVOKE

- Syntaxe

```
REVOKE [GRANT OPTION FOR] (privilèges | ALL)
ON liste d'objets
FROM liste d'utilisateurs
[{RESTRICT | CASCADE}]
```

- avec :
  - **CASCADE** : la révocation concerne les utilisateurs cités dans la clause FROM ainsi que ceux à qui ces privilèges ont été récursivement transmis.
  - **RESTRICT** : la révocation ne concerne que les utilisateurs cités dans la clause FROM.
  - **GRANT OPTION FOR** : ce n'est pas les privilèges qui sont révoqués, mais le droit de le transmettre.

Règle d'octroi des privilèges

- Un utilisateur ne peut octroyer que les privilèges qu'il possède

Règles de révocation des privilèges

- Un utilisateur ne peut révoquer que les privilèges qui lui ont été transmis
- Si l'option **CASCADE** est spécifiée, la révocation d'un privilège est récursive.
- Si l'option **RESTRICT** est spécifiée, la révocation d'un privilège à un utilisateur n'est possible que si celui-ci n'a pas transmis ce privilège à un autre utilisateur

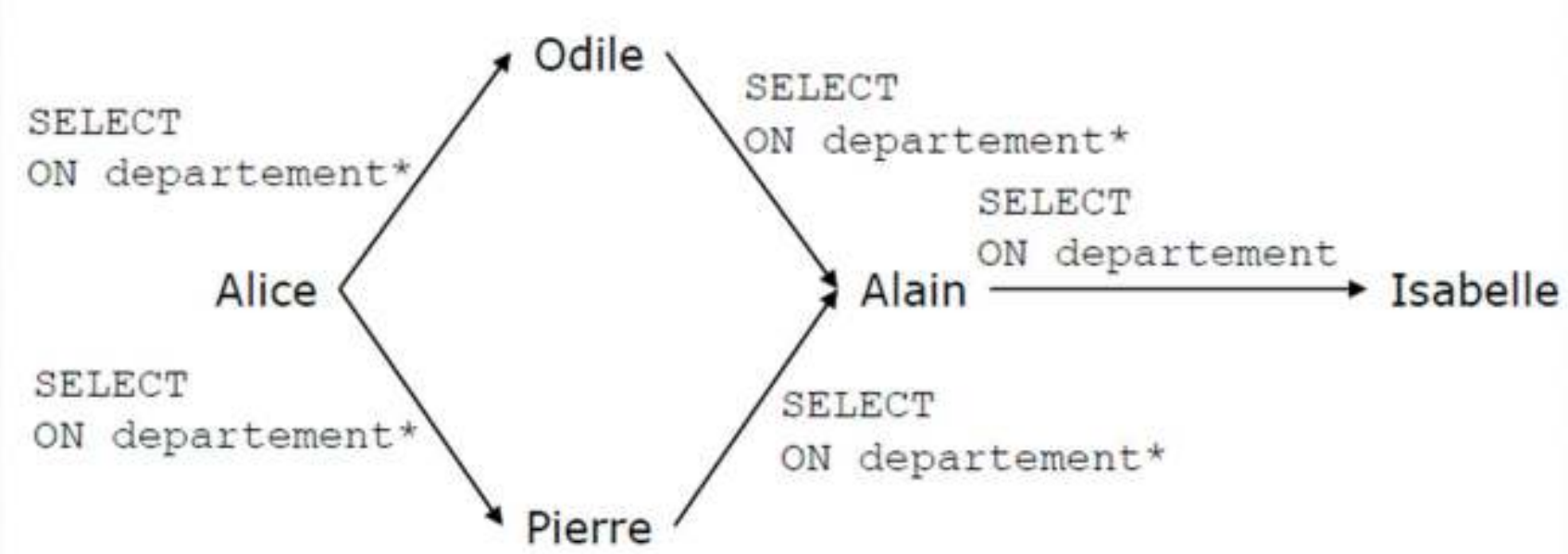
- Si un utilisateur a reçu un privilège de plusieurs utilisateurs, il ne perd ce privilège que si tous ces utilisateurs le lui retirent.

Graphe d'octroi des privilèges

- C'est un outil pratique pour gérer la propagation des privilèges et leur révocation.
  - chaque nœud représente un utilisateur, il est étiqueté par le nom de cet utilisateur,
  - chaque arc représente un octroi de privilège, il est étiqueté par ce privilège.

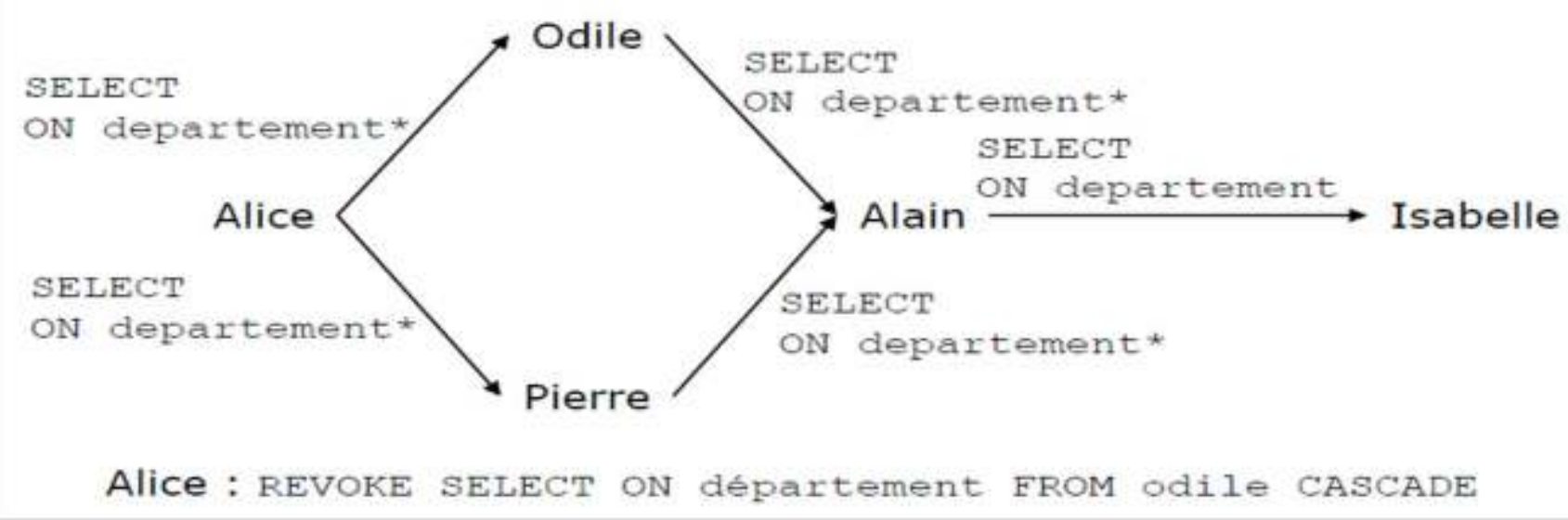
Octroi de privilèges : exemple

- `Alice : GRANT SELECT ON departement TO odile WITH GRANT OPTION;`
- `Alice : GRANT SELECT ON departement TO pierre WITH GRANT OPTION;`
- `Odile : GRANT SELECT ON departement TO alain WITH GRANT OPTION;`
- `Pierre : GRANT SELECT ON departement TO alain WITH GRANT OPTION;`
- `Alain : GRANT SELECT ON departement TO isabelle;`

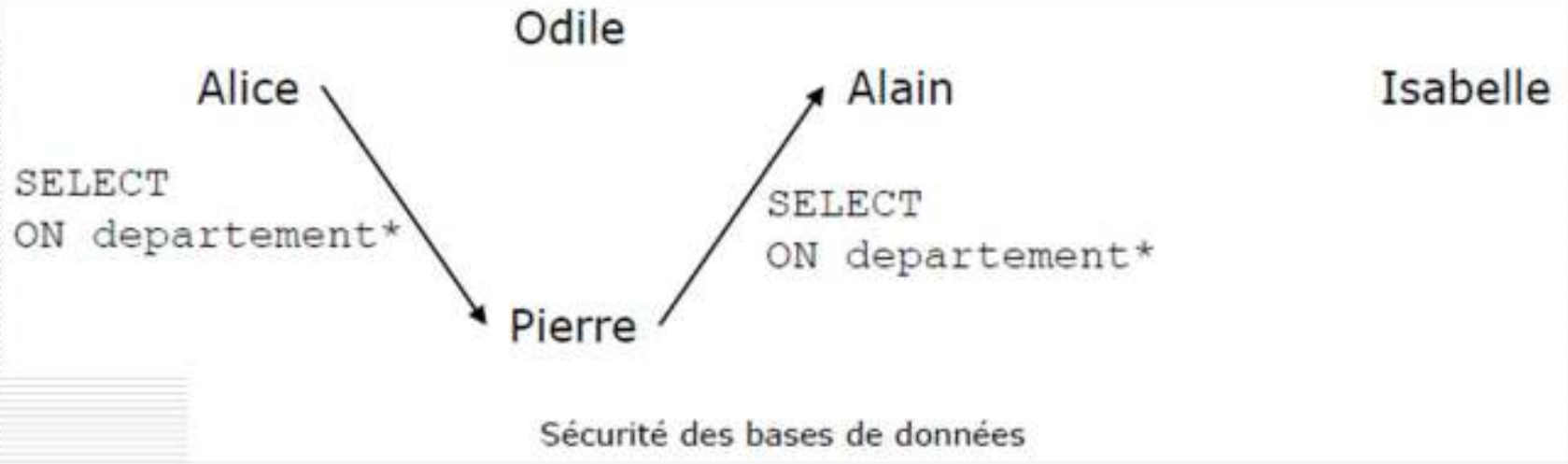


- `*` indique que le privilège a été accordé `WITH GRANT OPTION`.

Révocation en cascade : exemple



Alice : REVOKE SELECT ON département FROM odile CASCADE



Mesures de sécurité clés (1)

- **Gestion des accès :** Mettre en place des systèmes pour contrôler qui a accès aux données et à quels niveaux, par exemple, en utilisant des permissions basées sur les rôles

- **Chiffrement :** Transformer les données en un format illisible à l'aide de clés de chiffrement pour les protéger, que ce soit au repos ou en transit

- **Audit et surveillance :** Enregistrer et surveiller les activités sur la base de données pour détecter les comportements suspects ou les tentatives d'accès non autorisées.

Mesures de sécurité clés (2)

- **Sauvegarde et récupération :** Établir des procédures régulières de sauvegarde des données et des plans pour les restaurer en cas de perte ou de défaillance du système

- **Sensibilisation :** Former les employés aux bonnes pratiques de sécurité, car les erreurs humaines sont une cause fréquente de violations de données

- **Conformité :** S'assurer que les pratiques de sécurité respectent les réglementations en vigueur, comme le RGPD, qui établit des principes stricts pour la protection des données personnelles.

Exemples SQL pour chaque mode de contrôle d'accès

DAC - Discretionary Access Control

- Le propriétaire d'une table accorde des permissions directement
- **Créer une table**

```
CREATE TABLE Clients (
    id INT PRIMARY KEY,
    nom VARCHAR(50),
    portefeuille FLOAT,
    niveau_confidentialite INT
);
```

- **Donner à un utilisateur l'accès en lecture (SELECT)**

```
GRANT SELECT ON Clients TO userSara;
```

- **Donner l'accès en lecture et écriture**

```
GRANT SELECT, INSERT, UPDATE ON Clients TO userAli;
```

- **Retirer des droits de mise à jour des données d'une table**

```
REVOKE UPDATE ON Clients FROM userAli;
```

RBAC - Role-Based Access Control

- Les permissions sont attribuées aux **rôles**, puis les utilisateurs reçoivent ces rôles
- **Créer un rôle**

```
CREATE ROLE Analyste;
```

- **Attribuer des permissions à ce rôle**

```
GRANT SELECT ON Clients TO Analyste;
GRANT SELECT ON Transactions TO Analyste;
```

- **Attribuer un role à un utilisateur**

```
GRANT Analyste To UserMehdi
```

- **Retirer un rôle à un utilisateur**

```
REVOKE Analyste FROM userMehdi;
```

MAC – Mandatory Access Control (simulé en SQL) (1)

- MAC **n'est pas natif** dans SQL standard, mais se fait via **politiques de sécurité + labels** (ex. Oracle Label Security)

- **Créer une politique de classification** (Oracle Exemple)

```
BEGIN
  SA_SYSDBA.CREATE_POLICY(
    policy_name => 'Client_Policy',
    column_name => 'niveau_confidentialite'
  );
END;
```

- **Assigner un niveau de sécurité à un utilisateur**

```
EXEC SA_COMPONENTS.SET_USER_LABEL('userSara', 'CONFIDENTIEL');
```

MAC – Mandatory Access Control (simulé en SQL) (2)

- **Assigner un label de sécurité à une ligne**

```
INSERT INTO Clients (id, nom, portefeuille, niveau_confidentialite)
VALUES (1, 'Hatem', 100000, 'SECRET');
```

- Un utilisateur avec label *CONFIDENTIEL* **ne pourra pas** lire une ligne *SECRET*.

ABAC - Attribute-Based Access Control (1)

- Les droits dépendent des attributs (poste, heure, département…).
- En SQL on utilise :
  - des **vues filtrées**,
  - ou des **policies Row-Level Security** (PostgreSQL)

- **Exemple ABAC avec PostgreSQL RLS (Row Level Security)**
  - Autoriser un utilisateur à voir uniquement les clients de son département

    1) **Activer RLS**

```
ALTER TABLE Clients ENABLE ROW LEVEL SECURITY;
```

ABAC - Attribute-Based Access Control (2)

2. **Créer une politique basée sur un attribut département**

```
CREATE POLICY clients_dept_policy
ON Clients
FOR SELECT
TO userSalma
USING (departement = current_setting('app.user_departement'));
```

3. **Définir dynamiquement le département de l'utilisateur**

```
SET app.user_departement = 'Finance';
```

➡ L'utilisateur ne voit **que les lignes** dont le **département = Finance**.

Rule-Based Access Control (1)

- Accès contrôlé par des **règles globales** appliquées automatiquement
- Exemple : **Bloquer l'accès en dehors des heures de bureau**

```
CREATE OR REPLACE FUNCTION check_work_hours()
RETURNS BOOLEAN AS $$
BEGIN
  IF EXTRACT(HOUR FROM NOW()) BETWEEN 8 AND 18 THEN
      RETURN TRUE;
  ELSE
      RETURN FALSE;
  END IF;
END;
$$ LANGUAGE plpgsql;
```

Rule-Based Access Control (2)

- Appliquer cette règle à une table (PostgreSQL RLS)

```
CREATE POLICY work_hours_policy
ON Transactions
USING (check_work_hours());
```



En dehors de 8h-18h, l'accès est refusé.